MARI Quadcopter Drone

Connor Lee Prior
Department of Engineering
CSU East Bay
Hayward, USA
connorprior2000@gmail.com

Christopher Ruiz-Guerra
Department of Engineering
CSU East Bay
Hayward, USA
cruizguerra@horizon.csueastbay.edu

Nahom Solomon
Department of Engineering
CSU East Bay
Hayward, USA
nsolomon@horizon.csueastbay.edu

Seth Iris Canonigo
Department of Engineering
CSU East Bay
Hayward, USA
scanonigo@horizon.csueastbay.edu

Abstract*—The MARI drone group developed subsystems that work together to achieve a flight controller. We created software that reads data from a BNO055 inertial measurement unit (IMU) and a BMP390 barometer to extract roll, pitch, yaw, temperature, and relative altitude over I2C. We selected an STM32F756BG microcontroller running at its maximum clock speed of 216 MHz, paired with a proportional–integral–derivative (PID) control loop. The system makes real-time adjustments based on the user's PWM inputs from a radio transmitter and outputs PWM signals that drive the electronic speed controllers (ESCs) and motors. We developed a printed circuit board (PCB) with three dedicated power domains, using a switching regulator to step 14.8 V down to 5 V and a low-dropout (LDO) regulator to step 5 V down to 3.3 V. We then used a long-range radio (LoRa) module to wirelessly transmit data over more than a kilometer; it communicates with the microcontroller over a UART connection using AT commands. This allows live monitoring of IMU, BMP390, and PID data. We built a MARI LoRa Monitor application using the PyQt6 framework to parse LoRa packets and update a live graphical user interface (GUI) with telemetry. Despite not yet achieving stable flight, we have validated power in all domains and established PID communication on the three axes. We also overcame catastrophic hardware failures, including the burning of three boards. These failures helped the team learn the value of power-system planning, circuit design, hardware bring-up, hardware validation, firmware programming on microcontrollers, PCB and schematic design, and—most importantly—the value of working together as a team.*

Keywords*—quadcopter, UAV, PID control, STM32, FreeRTOS, LoRa telemetry, PCB design, IMU, BNO055, BMP390*

I. INTRODUCTION

For our Senior Design project, we decided to create a quadcopter drone. This project posed multiple design challenges that helped us better understand both the circuit board design and the programming required to build a drone that can actually fly. Instead of using a prebuilt system that we could simply plug in and use out of the box, we had to design our own printed circuit board. This introduces problems for power distribution to sensitive components, interpreting communication signals through protocols such as I2C and UART, wirelessly transmitting data into a human-readable format, and choosing parts that meet the design criteria we set out to reach.

Because a quadcopter is an inherently unstable system, another major component of this project was stabilizing the drone by making high-frequency adjustments to the motors to maintain its position. This involves using a proportional–integral–derivative (PID) control loop to make microadjustments to the motor outputs.

In this paper we describe our design process, the problems we encountered, and how we solved them.

II. DESIGN GOALS

In selecting features for our quadcopter drone, the primary goals were that it be remote-controlled, self-balancing, and extensible — i.e., the user should be able to add additional modules onto the drone if desired. To meet these goals, we first needed a microcontroller so that the propellers could be controlled by software. For remote control, we used a radio controller that sends radio signals from the user to a receiver on the drone. For self-balancing, we needed an inertial measurement unit (IMU), which detects the angular acceleration of the drone in pitch, roll, and yaw — values that are essential for stabilization. We also added a barometer to allow software-imposed altitude limits. For telemetry, we used a long-range radio (LoRa) module to wirelessly transmit data mid-flight. Finally, we ensured that we had many extra accessible pins on the microcontroller so that additional components could be added later.

III. SYSTEM BLOCK DIAGRAM

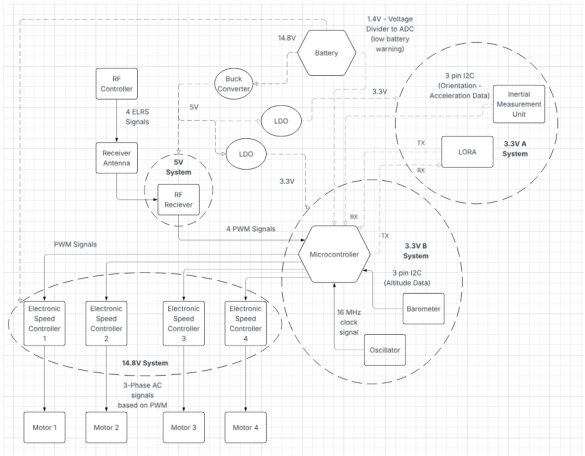

Fig. 1. System block diagram showing voltage domains and signal paths.

Our project has several subsystems that work together to get the drone flying. The first is the microcontroller — the brain of the system. We chose the STM32F756BG. By

programming the microcontroller, we use its logic to control the motor outputs based on user inputs and IMU data. We also connected the microcontroller to a crystal oscillator to run its clock at the maximum supported frequency of 216 MHz.

The two main sensors used in the project are the BNO055 IMU and the BMP390 barometer. The IMU sends acceleration and orientation data to the microcontroller. To track linear acceleration, the IMU contains a MEMS (micro-electro-mechanical systems) accelerometer, which works by tracking the movement of a small mass connected by springs to silicon plates. The mass forms a capacitor with the plates, and that capacitance can be measured. As the mass accelerates in any direction, the distance between mass and plate decreases and the capacitance in that direction increases. By tracking these capacitance differences, the accelerometer measures acceleration in any direction. The gyroscope uses a similar mass-and-plate system but measures angular velocity. Finally, the BNO055 includes a magnetometer that detects changes in the magnetic field by measuring voltage changes due to the Lorentz force on a current-carrying conductor.

These three sensors are combined natively on the BNO055 using its sensor-fusion algorithm, which acts as a Kalman filter — a method that improves accuracy by using data from multiple sensors to correct errors in any single sensor. A Kalman filter first defines a state vector containing the variables to be tracked. It then predicts the next state using a mathematical model (analogous to using speed and time to compute distance traveled). The algorithm compares its prediction with the next measurement and combines the two values according to a Kalman gain. The fused orientation values are then sent to the microcontroller, which adjusts the motor outputs so the drone self-corrects when out of position. The IMU communicates with the microcontroller over I2C.

For user control, we used a prebuilt radio controller that sends radio signals to a receiver, which in turn sends pulse-width modulated (PWM) signals to our microcontroller. We use four channels so the user can control throttle (up/down), pitch (tilt forward/back), roll (tilt left/right), and yaw (turn left/right). This allows the drone to move freely in three dimensions.

Once the microcontroller has interpreted the user inputs and adjusted them based on IMU measurements, the next step is to convert them into motor commands. These are also PWM signals, sent to our four electronic speed controllers (ESCs). The ESCs act as an interface between the microcontroller and the motors: the microcontroller is a sensitive low-voltage, low-current device, while the motors require high voltage and current. The ESCs bridge this gap by interpreting the PWM signals into the three-phase power that the motors need to spin.

All of this is supported by our power system, which is split into three voltage domains: 3.3 V, 5 V, and 14.8 V. The 14.8 V domain is supplied directly by the battery. We then drop this voltage to 5 V using a buck (switching) regulator, and from 5 V to 3.3 V using a low-dropout (LDO) regulator.

IV. SCHEMATICS

A. Power System

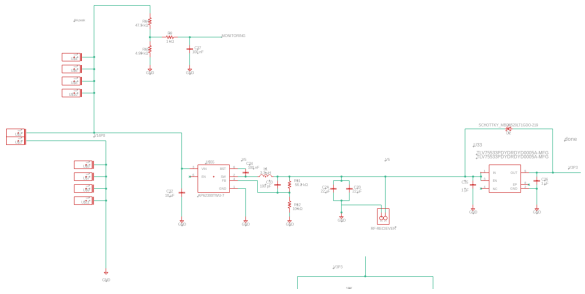

Fig. 2. Power system schematic.

The MARI drone power system uses a single battery to support multiple voltage domains. The system can operate on either a 3S or 4S LiPo battery, with nominal voltages of 11.1 V and 14.8 V respectively. Supporting both configurations provides flexibility, but also requires every component to be compatible with the full voltage range. Power calculations were performed using the nominal 4S voltage of 14.8 V.

The battery directly powers the four ESCs and motors, which require the largest share of total power in the system. Lower-voltage components are supplied through regulators: after the ESCs and motors, the battery voltage is stepped down to 5 V using a buck converter, and then to two separate 3.3 V domains using LDOs for sensitive components such as the microprocessor and sensors.

To calculate total system demand, the power of each subsystem was calculated using $P = V \times I$.

$I_{motor} = 23.232$ A (per motor)

$I_{motors,total} = 4 \times I_{motor} = 92.928$ A

$I_{monitoring} = 0.322$ mA

$P_{14.8V} = V_{batt} \times (I_{motors,total} + I_{monitoring}) = 1375.33$ W

For the regulated subsystems, the 5 V and 3.3 V rails are calculated as:

$P_{5V} = 5.0\text{ V} \times I_{5V,total} = 3.364$ W

$P_{3.3VA} = 3.3\text{ V} \times I_{3.3VA,total} = 0.537$ W

$P_{3.3VB} = 3.3\text{ V} \times I_{3.3VB,total} = 1.081$ W

The total system power is $P_{total} = P_{14.8V} + P_{5V} + P_{3.3VA} + P_{3.3VB} \approx 1380.32$ W.

The motors and ESCs accounted for most of the power consumption, while the 5 V and 3.3 V subsystems consumed significantly less but remained critical for system control. To improve voltage stability and reduce noise, LC filtering and bypass capacitors were placed throughout the design. During initial testing the system was prone to voltage spikes from the battery, which we had not initially accounted for. This was resolved by adding anti-spark XT90 connectors at the battery terminals and a bulk capacitor at the PCB input to smooth the input voltage and reduce transient effects. A battery-monitoring circuit was also included to measure the voltage and provide LED feedback when voltage levels approached unsafe conditions.

B. Microcontroller

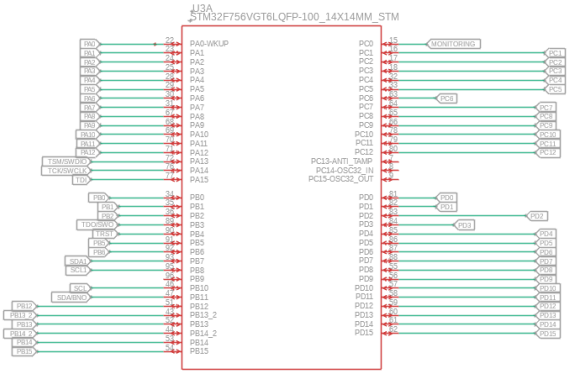

Fig. 3. Microcontroller schematic — peripheral connections.

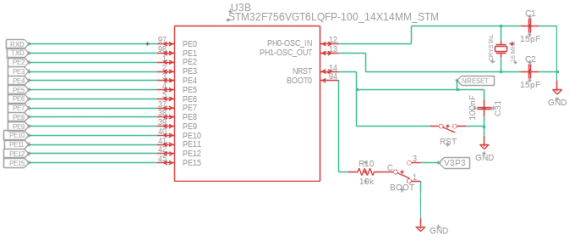

Fig. 4. Microcontroller schematic — BOOT and oscillator circuit.

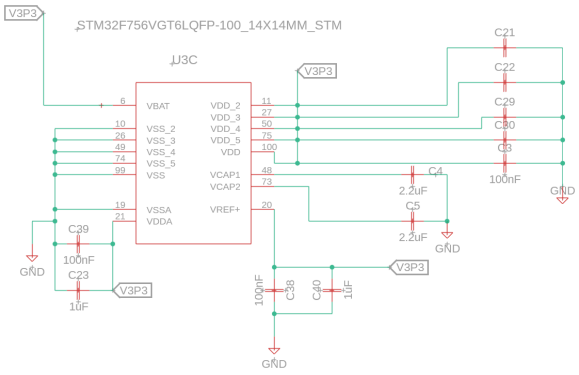

Fig. 5. Microcontroller schematic — bypass capacitors and power.

Because the microcontroller is connected to every other system, most of its pins are used for connections to other subsystems or to header pins for optional add-ons. Diagram 1 (Fig. 3) consists primarily of these connections, while Diagram 2 (Fig. 4) also includes our BOOT circuit and crystal oscillator. Diagram 3 (Fig. 5) includes the bypass capacitors used for the microcontroller and the main connections to the power system.

Although it was not strictly an error, we initially misunderstood the function of the BOOT circuit in this schematic. We thought it would turn the microcontroller on or off by removing power; in reality, it switches one of the operating modes of the microcontroller.

C. Inertial Measurement Unit

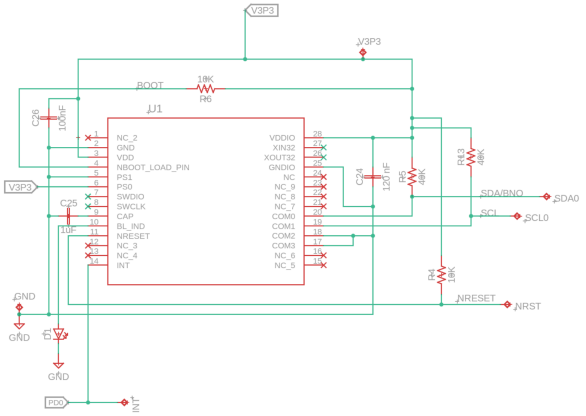

Fig. 6. IMU (BNO055) schematic.

For our inertial measurement unit, we used the recommended schematic from the BNO055 datasheet. This involves connecting the I2C lines (SDA, SCL, INT) to the microcontroller, supplying power from the 3.3 V domain, and including pull-up resistors and bypass capacitors where required.

One error we encountered in this schematic was that the PS0 pin was supposed to be tied to GND but was instead connected to 3.3 V. This unintentionally changed the communication mode from I2C to HID-over-I2C, which made the part effectively impossible to communicate with using our existing tooling. We attempted to repair the issue with rework but found it too difficult to fix in place. Instead, we used the spare pins on our microcontroller to connect to an external BNO055 module, preserving full IMU functionality.

D. Barometer

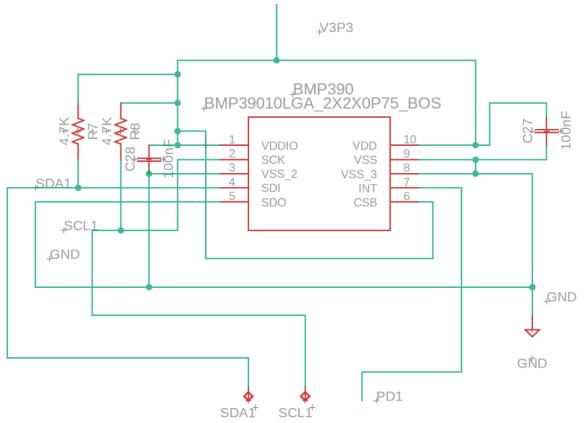

Fig. 7. Barometer (BMP390) schematic.

The barometer schematic is very similar to the IMU schematic. We followed the recommended schematic from the BMP390 datasheet, connected the three I2C lines to the microcontroller, and supplied the part from the 3.3 V domain.

V. LORA TELEMETRY

To monitor the drone during flight testing and PID tuning, we needed a way to wirelessly transmit sensor data, motor outputs, and system status from the drone back to a ground station. We chose LoRa as the wireless link because it offers long range — over a kilometer in open conditions — at very low transmit power, and uses an inexpensive, easy-to-integrate module (the REYAX RYLR998) that communicates with the microcontroller over a simple UART connection using AT commands.

However, LoRa has one major drawback for telemetry: the available bandwidth is very limited. The core physical-layer transmission parameters for LoRa are spreading factor, bandwidth, coding rate, and preamble length, and they determine the tradeoff between range, robustness, throughput, and airtime. A higher spreading factor lowers the symbol rate, increases receiver sensitivity, and extends communication range, but it also makes each packet take longer to transmit. The same general tradeoff applies to bandwidth: a narrower bandwidth improves RF sensitivity (which increases range) but increases transmission time. Coding rate adds forward error correction, improving robustness against data loss at the cost of overhead, and a longer preamble can improve reception reliability against noise but also adds airtime [4], [5].

For our settings, we chose a spreading factor of 7, 125 kHz bandwidth, 4/5 coding rate, and 8 preamble symbols; with these parameters a single packet takes between 45 and 75 ms to transmit depending on payload size. This was determined using the tested throughput in the datasheet [1]. Designing the telemetry system was therefore mostly an exercise in deciding what data was important enough to send, how often to send it, how to format it efficiently, and how to make tradeoffs between range, noise, and transmission time.

One side of the link consists of the STM32 connected to a RYLR998 module by UART; the other side consists of a PC connected to another RYLR998 module via a USB-UART TTL adapter. The module handles all LoRa physical-layer details internally and exposes a text-based AT command interface to the host. To send a message, the host writes an AT+SEND command to the module specifying destination address, payload length, and ASCII data. When a message is received on the other end, the receiving module emits a line of the following form on its UART:

+RCV=<address>,<length>,<data>,<RSSI>,<SNR>
[2]

The five fields contain the sender's address, the number of bytes in the payload, the actual data, the received signal strength indicator (RSSI, in dBm), and the signal-to-noise ratio (SNR, in dB). For example, a real line received during a test flight looks like:

+RCV=1,28,I:1245|0.38,-6.19,5.00,26.8,996.9,-65,8

The RSSI and SNR fields are added by the receiving module itself based on the radio conditions when the packet arrived. They give a real-time measure of link quality independent of the message contents, which is useful for telling whether dropped packets are due to range issues, interference, or other causes.

The LoRa modem provides a transparent byte-level link, but it has no notion of what kind of data we are sending. To distinguish, for example, IMU data from motor outputs, and to detect packets lost in transit, we designed our own message format that lives inside the data field of each LoRa packet:

TYPE:SEQ|PAYLOAD

The type tag is a short identifier for what type of data the packet contains. The sequence number is an integer that increments by one for each message of that type. The payload is a comma-separated list of values whose meaning depends on the type. For example, an IMU reading is sent as:

I:124|0.38,-6.19,5.00,26.8,996.9

This says: message type I (IMU and barometer), sequence number 124, with five payload values in fixed positions (pitch in degrees, roll in degrees, yaw in degrees, temperature in °C, and pressure in hPa).

TABLE I. LORA TELEMETRY MESSAGE TYPES

Type	Meaning	Payload contents
I	IMU and barometer	pitch, roll, yaw, temp., pressure
M	Motor mix PWM	front-L, front-R, rear-L, rear-R outputs
P	PID terms	pitch P, I, D; roll P, I, D; yaw P, I, D
B	Battery status	NORMAL | WARNING | CRITICAL
R	Task rates	flight, tele tx, tele rx, housekeeping
C	CPU usage	flight, tele tx, tele rx, hk, idle
RW	Raw text	Any string
MARI	Special ping reply	Fixed 88-byte ping response

We chose this format because it is easy for the receiving link, which is connected to our Python application, to parse and sort data efficiently by message type. By including an incrementing sequence number, the TYPE:SEQ pair effectively serves as a pseudo-UUID, making detection of missing or duplicate packets easier to debug.

One challenge of LoRa telemetry is finding the correct parameters and balancing transmission airtime against range, robustness, and reliability. Because of the number of packets being sent per second and the relatively close range during development, we ended up sacrificing distance for lower transmission time, which is why we chose the parameters listed above. The exceptions are bandwidth and coding rate, neither of which is worth changing for the small reduction in airtime they would provide. As shown in Fig. 8, reducing the

spreading factor from the default of 9 to 7 greatly reduces transmission time per payload byte.

$$N_{payload}(PL, SF, CR) := 8 + \max\left[\left\lceil \frac{8 \cdot PL - 4SF + 28 + 16CRC - 20IH}{4(SF - 2DE)} \cdot (CR + 4) \right\rceil, 0\right]$$

$$T_{packet}(PL, SF, CR, n_{preamble}, BW) := (n_{preamble} + 4.25) \cdot \frac{2^{SF}}{BW} + N_{payload}(PL, SF, CR) \cdot \frac{2^{SF}}{BW}$$

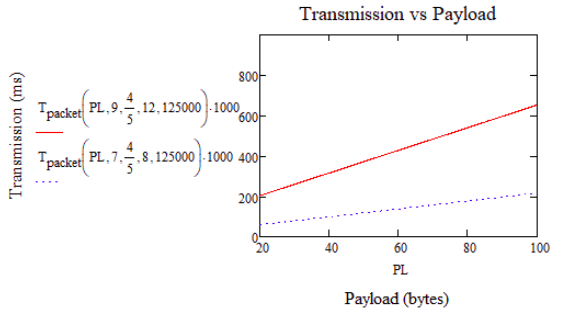

Fig. 8. Transmission time vs. payload size for SF7 and SF9 (BW = 125 kHz, CR = 4/5).

VI. PCB LAYOUT 1

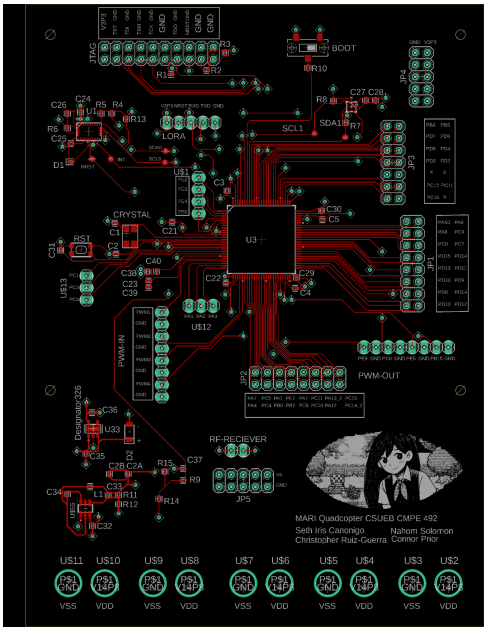

Fig. 9. PCB Layout 1 — full board.

Our PCB holds the power system, microcontroller, IMU, and barometer, and is split by voltage domain: the top is the 3.3 V domain, the bottom is the 14.8 V domain, and a small space in the middle is the 5 V domain. Our overall strategy was to keep sensitive components close to the components they communicate with while keeping voltage domains as clear and separate as possible.

The PCB is a four-layer board. The top layer carries the red traces, the two inner layers are dedicated VDD and ground planes, and the bottom layer carries the blue traces. Having a bottom routing layer makes it much easier to lay out signals because traces can effectively cross one another without shorting.

A. Microcontroller, IMU, Barometer, and LoRa

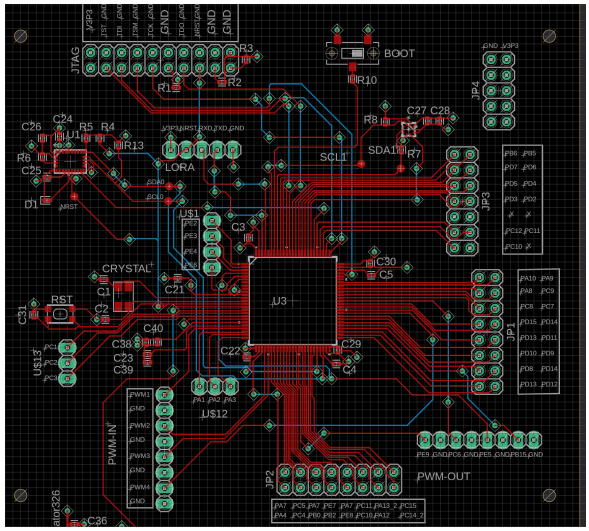

Fig. 10. Microcontroller, IMU, barometer, and LoRa layout.

The microcontroller is the hub of the board, with connections to almost every other component on the PCB. The traces follow the schematic exactly, allowing us to lay out the components based directly on the schematic. This layout also shows the many extra microcontroller pins available for connecting to optional external components later. We added identifiers to these pins on the silkscreen, since constantly cross-referencing the schematic to identify pins would have been impractical. Of particular note are the JTAG pins on the top of the board, which are used to program the microcontroller.

The IMU, LoRa, and barometer layouts follow their schematics precisely. However, because the pads of these components sit underneath the parts themselves, we added test points to almost all traces coming out of the IMU; without them they would become impossible to access during testing if something went wrong.

B. Power System

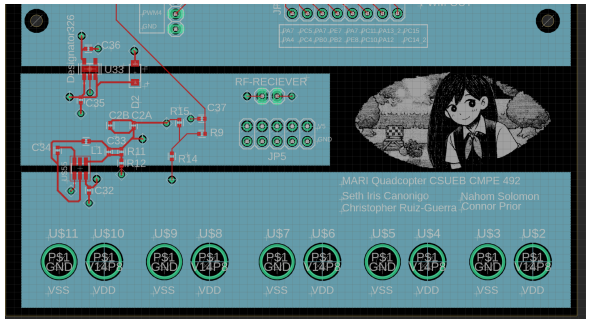

Fig. 11. Power system layout (bottom of board).

The power system bridges between clearly defined voltage domains. The bottom of the board contains the 14.8 V domain. The through-holes there are intended for bullet connectors that connect to five components: the 14.8 V battery and the four ESCs. The layout clearly shows the 14.8 V domain connected to the 5 V domain through the buck

converter, and the 5 V domain connected to the 3.3 V domain through the LDO.

One error we encountered in this layout was the indicators for the Schottky diode. Because we were not fully familiar with the conventions for labeling the anode and cathode of a diode in our footprint, the part ended up being placed in reverse during assembly. This had devastating consequences and resulted in two unusable boards. Once we identified the problem the fix was straightforward: switching the diode's orientation made the system work as intended.

VII. PCB LAYOUT 2

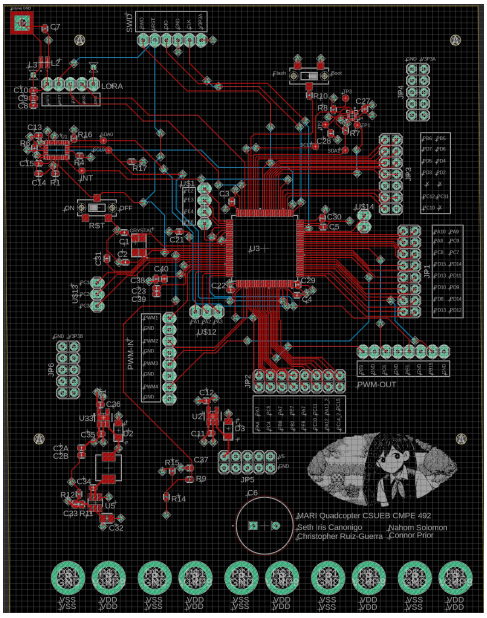

Fig. 12. PCB Layout 2 — revised board.

After burning multiple boards and having trouble with the IMU connection, we did a new layout to fix the old problems. The main changes are a large bulk capacitor in the 14.8 V domain and a larger inductor after the buck converter to handle higher voltages. We also cleaned up some of the pins we determined were unnecessary and corrected the IMU connection.

VIII. SOFTWARE

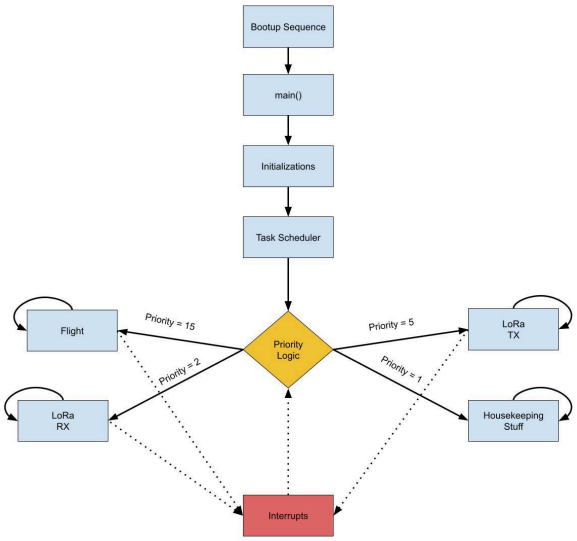

Fig. 13. FreeRTOS task structure overview.

While the software was initially structured as a single continuous control loop, it now runs on a real-time operating system (RTOS), specifically FreeRTOS, because the different parts of our software have different priorities and very different timing requirements. The flight control has a hard real-time deadline — if it does not run fast enough, the drone can crash. Other components such as telemetry, barometer reading, and battery monitoring are less critical and can even bottleneck the flight calculation if not properly prioritized. An RTOS lets us guarantee that the time-critical code always runs when it needs to, even if a slower task is in the middle of doing something else. With the RTOS, the software is divided into four main tasks, each running at its own rate and priority: FlightTask, TelemTxTask, TelemRxTask, and HousekeepingTask.

A. *FlightTask*

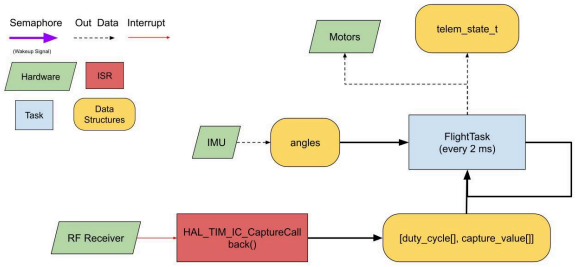

Fig. 14. FlightTask data flow.

The most important task is FlightTask, which runs the flight control loop at 500 Hz. This means it has only 2 ms to read the IMU, compute pitch, roll, and yaw errors, run three PID controllers, apply the mixing equations, and update the four motor PWM outputs. Because it has the highest priority, it preempts whatever lower-priority task is currently running.

First, we configure timers to capture the PWM signals from the radio receiver and to drive the PWM outputs to the ESCs.

We also initialize the sensor configuration registers to ensure the sensors are set up correctly. We then enter the control loop.

To start each iteration, we read the user input via timer capture to determine the duty cycle of each PWM channel. Next, we read data from the registers of the IMU and barometer. We then calculate the error between the user input setpoint and the measured orientation from the IMU. The PID equations are then applied to those errors to control the system.

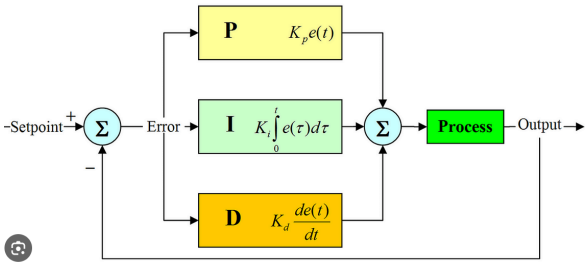

Fig. 15. PID control loop.

The idea of a PID controller is to take the error and multiply it by each of the three terms — proportional, integral, and derivative — each weighted by a manually-tuned gain. The proportional term lets the drone react quickly to error. The integral term, which sums error over time, lets the drone correct steady-state offsets that the proportional term alone cannot eliminate. The derivative term, which acts on the rate of change of error, helps curb overshoot.

Finally, mixing equations turn the controller outputs into PWM commands for each motor according to throttle, roll, pitch, and yaw:

Motor1 (Front Left) = throttle - roll - pitch + yaw
Motor2 (Front Right) = throttle + roll - pitch - yaw
Motor3 (Rear Right) = throttle + roll + pitch +yaw
Motor4 (Rear Left) = throttle - roll + pitch - yaw
This completes the control loop, and the cycle repeats.

B. *TelemTxTask*

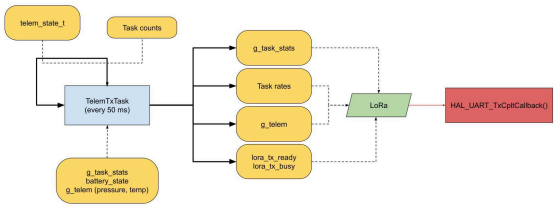

Fig. 16. TelemTxTask data flow.

The second-most important task is TelemTxTask, which decides what to send and when. It runs at 20 Hz, woken every 50 ms by tick polling. Each iteration it does three things: take a snapshot of the latest data from the HousekeepingTask and FlightTask, prepare it for the message scheduler, and call into the message scheduler to actually send something to UART using the AT+SEND command. Instead of blindly transmitting every message every cycle, which would massively exceed the LoRa link's capacity, the scheduler implements a round-robin over message types with two layers of gating: an "+OK" check and per-message-type rate limiting.

Because every AT+SEND issued from the host is acknowledged by an "+OK" line from the module, the scheduler always waits until it receives that response or until a one-second timeout expires. This prevents overrunning the LoRa modem. Because some message types are more important than others — for example, IMU data versus CPU usage — message types also have priorities relative to one another, so the scheduler sends the higher-priority types more often than the lower-priority ones.

C. *TelemRxTask*

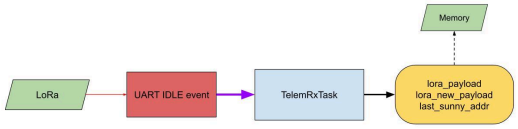

Fig. 17. TelemRxTask data flow.

TelemRxTask is the second-least important task. Unlike the other three tasks, it is fully interrupt-driven. When the ground station finishes sending a line over UART, a hardware "idle line" interrupt fires, which releases a semaphore that wakes TelemRxTask to parse the message. This keeps the time between a command arriving and the drone responding to a few microseconds, instead of polling the UART in software. Currently only one command is implemented: pinging the drone.

D. *HousekeepingTask*

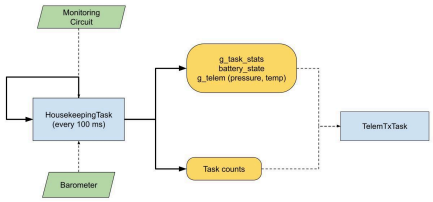

Fig. 18. HousekeepingTask data flow.

The lowest-priority "default" task is HousekeepingTask. Unlike the other three tasks, it is fully periodic, running every 100 ms when neither FlightTask nor TelemTxTask is running. It records the task rates and CPU usage, runs the barometer, runs the battery-monitoring logic, and writes the resulting values to the global variables that TelemTxTask reads.

One of our biggest challenges was configuring the polling rate of the fastest task without starving the slower tasks. This was the case for FlightTask, which was initially given a 1 ms polling period (1 kHz). Unfortunately, IMU read time plus PID computation exceeded 1 ms, which caused FlightTask to run essentially continuously, starving TelemTxTask and HousekeepingTask and preventing any data from being transmitted. Our solution was to roughly quadruple the I2C bus speed and double the FlightTask period.

IX. MARI LORA MONITOR (GROUND STATION)

To monitor the drone mid-flight, we developed a desktop Python application using the PyQt6 framework called the MARI LoRa Monitor. The application is connected to the second RYLR998 module through a USB-UART TTL device. It receives the +RCV messages, parses them into readable data according to the LoRa message-type table, and updates a live dashboard.

Fig. 19. MARI LoRa Monitor — Console tab (disconnected state).

Fig. 20. MARI LoRa Monitor — Telemetry tab (connected state).

X. RESULTS

While we have come a long way toward finishing the drone project, we still have a long way to go to complete it, owing in large part to the problems we encountered with the power system and the pin assignments in our PCB. We have nonetheless made significant progress: the power system is fully functional and the software is working as intended. Although the on-board IMU is no longer functional, we have used an external module that works correctly. The total power consumption of our system was approximately 1091.9 W. The total cost of the components on the PCB was $45.52; however, in practice each board cost about $100 to construct. The off-board components added up to about $330, bringing the total cost of an individual drone to approximately $430.

XI. CONCLUSION

Overall, there is plenty more to work on with our drone project. Although all of our subsystems are functional, we still need to tune the PID gains to achieve stable hover. One of the main lessons we learned was just how difficult the power system was to get right; a strong understanding of the fundamentals of circuits will always be very important when designing systems like this drone. We also learned how important step-by-step verification is — it is very hard to debug a system when you are not sure what the problem is. Finally, we learned how important it is to be able to see data while testing; it is much easier to debug when you can notice patterns in the data and form hypotheses based on them.

ACKNOWLEDGMENT

The authors would like to thank the faculty of the Department of Engineering at California State University, East Bay, for their guidance and support throughout this project.

REFERENCES

[1] Bosch Sensortec, BNO055 Intelligent 9-axis absolute orientation sensor, BST-BNO055-DS000-18, Oct. 2021.

[2] REYAX Technology Co., Ltd., RYLR998 UART Interface 868/915 MHz LoRa Antenna Transceiver Module Datasheet, 27 Feb. 2025, "Data Throughput Test," p. 10.

[3] REYAX Technology Co., Ltd., LoRa AT Command Guide: Apply for RYLR998, RYLR498, 7 Aug. 2025, "+RCV Show the received data actively," p. 8.

[4] F. Adelantado, X. Vilajosana, P. Tuset-Peiro, B. Martinez, J. Melià-Seguí, and T. Watteyne, "Understanding the limits of LoRaWAN," IEEE Communications Magazine, vol. 55, no. 9, pp. 34–40, 2017, doi: 10.1109/MCOM.2017.1600613.

[5] Semtech Corp., SX1272/3/6/7/8 LoRa Modem Design Guide, application note AN1200.13, Rev. 1, Jul. 2013, sec. 4, "The LoRa Packet Format & Time On Air," p. 7.